ISO/IEC/IEEE 29119-1:2022

# Software Testing Standards

*Sections 4.4 – 4.7 — Test Design, Project Integration, Communication, and Defect Management*


TEAM 2
SHIRLEY ARAGON
FERNANDO DIAZ
SALVADOR MUÑOZ
ANTHONY NAVARRETE
GABRIEL TUMBACO


OVERVIEW

# What this section of the standard covers

| | | |
|---|---|---|
| 4.4 | **Test Design & Execution** | Test models, testing approaches, experience-based techniques |
| 4.4 | **Modern Testing Practices** | Retesting, regression, automation, continuous testing |
| 4.5–4.6 | **Project Management & Communication** | Environments, test data, planning, stakeholder reporting |
| 4.7 | **Defect & Incident Management** | Investigating failures and closing the loop |

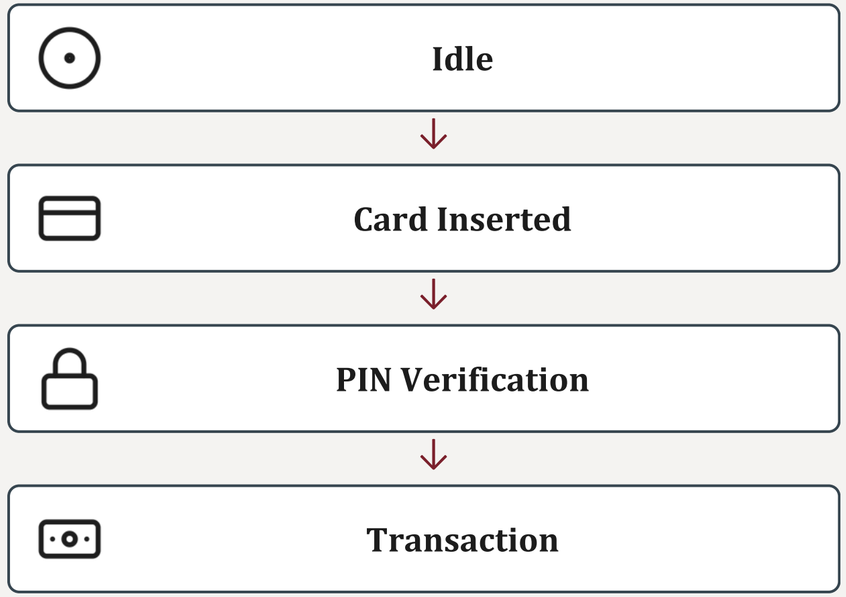
# The test model: an abstract map of expected behavior

A test model is not the software itself.

It is a simplified representation that captures the states, transitions, inputs, outputs and conditions relevant for testing.

- What states can the system be in?
- What events trigger a transition?
- Which inputs are valid or invalid?
- What output is expected in each case?

## EXAMPLE — ATM STATE MODEL




## SUGGESTED TECHNOLOGIES

### PlantUML / Enterprise Architect

Draw state and transition models directly from requirements.

### GraphWalker

Open-source model-based testing engine that walks a state graph.

### draw.io

Quick collaborative diagrams for decision tables and flows.

# Three ways to design and run a test

### Model-Based Testing

A behavior model (state diagram, decision table) is built first; test cases are then derived from it, manually or with a tool.

### Scripted Testing

Steps, inputs and expected results are fully prepared in advance and followed precisely during execution.

### Exploratory Testing

The tester learns the system while interacting with it, designing new test ideas in real time.

*ISO 29119 recommends combining these approaches rather than relying on a single one.*

## SUGGESTED TECHNOLOGIES

**Conformiq / GraphWalker**

Generate test cases automatically from a behavior model (MBT).

**Selenium / Cucumber (Gherkin)**

Author and run scripted, repeatable test steps.

**Rapid Reporter / Session Tester**

Log findings live during exploratory test sessions.

# Experience-based testing: knowledge as a method

Testers draw on domain expertise and lessons from prior projects rather than deriving tests purely from documentation.

**Tours**

The tester is guided through the important parts of the application, much like a tourist visiting a city's landmarks.

**Attacks**

Tests intentionally exploit likely weaknesses, e.g. invalid input or disconnecting a required service, to observe the system's reaction.

**Checklist-Based Testing**

Testers work through a checklist of common issues like error messages, UI consistency or input validation, instead of designing tests from zero.

## SUGGESTED TECHNOLOGIES

### OWASP ZAP / Burp Suite

Automate attack-style probing for common security weaknesses.

### TestRail / Xray Checklists

Maintain and execute reusable checklist-based test suites.

### Maze / UserTesting

Structured exploratory 'tours' through a live product with real users.

# Confirming fixes without breaking everything else

## RETESTING

***Verifies that a previously reported defect has actually been fixed.***

After the login bug is fixed, retesting confirms users can log in again.

## REGRESSION TESTING

***Verifies that new changes have not unintentionally broken existing functionality.***

Registration, password recovery and profile management are re-checked.

Manual testing is performed by people;

Automated testing uses tools to run the same cases repeatedly.

## SUGGESTED TECHNOLOGIES

### Selenium / Playwright / Appium

Automate regression and retest execution across web and mobile.

### Jenkins / GitHub Actions / GitLab CI

Re-run automated suites on every build or commit.

### Zephyr / TestRail

Track which cases need retesting after a defect is closed.

# Continuous testing and specialized techniques

Continuous testing runs automated tests automatically whenever the software changes — after every commit or new build — so defects surface much earlier.

### Back-to-Back Testing

Compares the outputs of two independent implementations of the same system.

### A/B Testing

Compares two versions of a feature and uses statistical evidence to see which performs better.

### Fuzz Testing

Automatically generates thousands of random or unexpected inputs to check the software doesn't crash.

## SUGGESTED TECHNOLOGIES

**Jenkins / CircleCI / GitLab CI**
Trigger test suites automatically on every commit (continuous testing).

**Optimizely / Google Optimize**
Run and measure A/B experiments with statistical confidence.

**AFL / libFuzzer / OSS-Fuzz**
Generate randomized inputs for automated fuzz testing.

# A reliable environment produces reliable results

A test environment includes the software, hardware, networks, databases, services, interfaces, testing tools and test data needed to run a test.

## TEST DATA MANAGEMENT

- Sensitive information should be anonymized where necessary.
- Test data should be version-controlled.
- This allows tests to be repeated under identical conditions.

## ENVIRONMENT COMPONENTS

- Software
- Hardware
- Networks
- Databases
- Services
- Interfaces
- Testing tools
- Test data

## SUGGESTED TECHNOLOGIES

### Docker / Kubernetes

Spin up consistent, disposable test environments on demand.

### Testcontainers

Provision real dependencies (DBs, queues) for integration tests.

### Mockaroo / Faker.js

Generate realistic, anonymized synthetic test data.

# Testing inside the project, reported to the right audience

Testing activities are planned together with project schedules, budgets and risk management.

## WHO NEEDS WHAT

| | |
|---|---|
| **Developers** | Defect details and required modifications |
| **Project Managers** | Progress against schedule, budget and testing status |
| **Other Stakeholders** | Completion and outcome of the testing effort |

## SUGGESTED TECHNOLOGIES

### Jira / Azure DevOps

Link test plans and defects directly to sprint and release schedules.

### Zephyr / Xray for Jira

Generate real-time test progress and coverage reports.

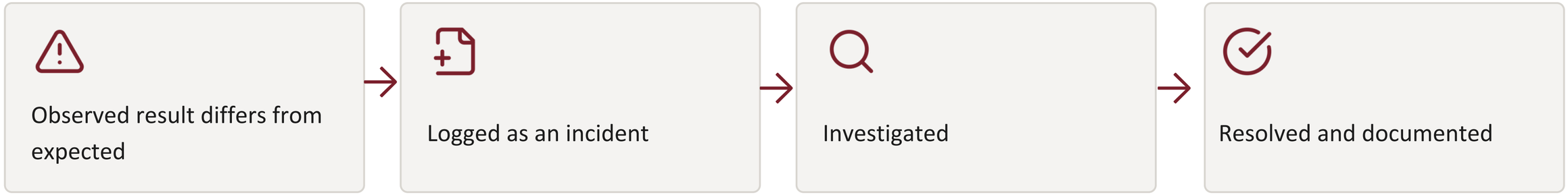
### Confluence / Power BI

Share tailored status reports with different stakeholder groups.

# A failed test is a question, not a verdict

A failed test does not automatically mean the software contains a defect. The failure must first be investigated.

The problem could originate from the software, an incorrect test case, or even an incorrect requirement.




*This systematic process improves software quality and strengthens collaboration between testers and developers.*

## SUGGESTED TECHNOLOGIES

### Jira / Bugzilla / MantisBT

Log, triage and track incidents through to resolution.

### Azure DevOps Boards

Link incidents back to the originating test case and requirement.

### Grafana / Allure Reports

Visualize defect trends and test-failure root causes over time.

CONCLUSION

# A structured framework for software testing

ISO/IEC/IEEE 29119 explains how to design effective tests, combine different testing techniques, integrate testing with project management, communicate results clearly, and manage defects systematically.

- Higher software quality
- Reduced project risk
- More reliable, predictable releases

---

*Thank you — questions welcome.*